

OpsPilot

Incident Response Copilot

AIOps · RAG · LangGraph · Qwen2.5:7B · H100 GPU

Complete Technical Guide — v1.0 | June 2026

Author: Adarsh Mishra

Model: Qwen2.5:7B (Q4_K_M) via Ollama on H100 GPU

Repo: github.com/adarshmishra121/OpsPilot

1. Project Overview

OpsPilot is an **AIOps incident response copilot** that combines anomaly detection, hybrid retrieval-augmented generation (RAG), and a tool-using LLM agent to automatically analyse production incidents, retrieve relevant runbooks, and generate grounded, evidence-backed remediation plans.

Key capabilities:

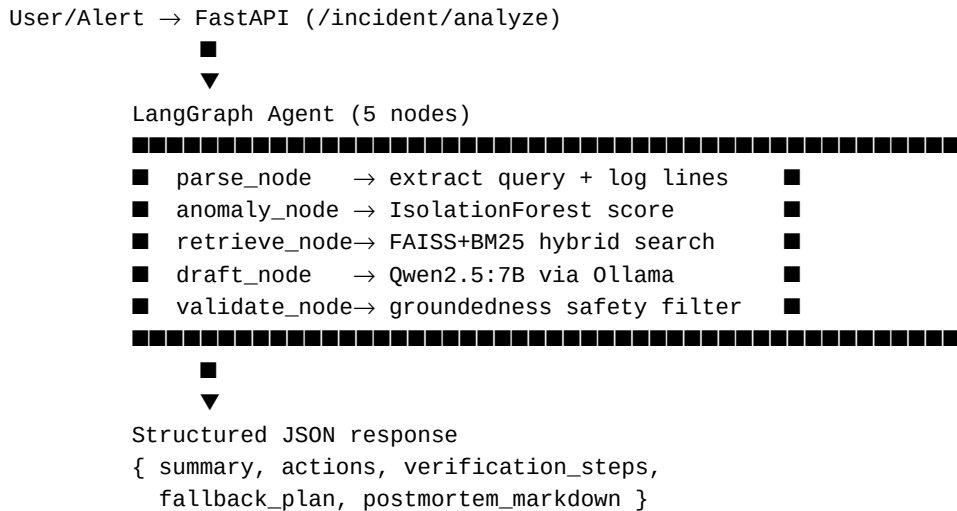
- Drain3 log template mining on the HDFS dataset (50 k+ lines)
- IsolationForest anomaly scoring with TF-IDF + statistical features
- Hybrid RAG: FAISS (semantic, 60%) + BM25 (keyword, 40%) over 200+ Kubernetes runbooks
- LangGraph 5-node state machine: parse → anomaly → retrieve → draft → validate
- Safety validator: rejects any action not grounded in a retrieved runbook
- FastAPI REST backend, Streamlit UI, Prometheus/Grafana observability, MLflow experiment tracking
- Full DVC-managed reproducible pipeline

Metric	Value
Training dataset	HDFS log dataset (50 k+ lines)
Runbook corpus	200+ Kubernetes runbooks
Anomaly model	IsolationForest (sklearn 1.7)
RAG index	FAISS + BM25, alpha=0.6
LLM	Qwen2.5:7B (Q4_K_M, Ollama)
GPU	H100 80 GB (private Docker container)
Inference speed	~800 tokens/s (GPU) vs ~1.5 tokens/s (CPU)
P95 latency	~1 000 ms end-to-end (GPU)
RAG MRR	0.59
RAG Recall@6	0.58
Anomaly pct	25% (contamination=0.01)

2. System Architecture

OpsPilot is structured as a layered system with a clear separation between the offline training pipeline and the online inference pipeline.

2.1 High-Level Flow



2.2 Training Pipeline (DVC)

Stage	Script	Output
download	scripts/data/download_all.py	data/raw/
parse	scripts/features/parse_logs.py	data/processed/templates.jsonl
features	scripts/features/build_features.py	artifacts/anomaly_vocab.json
train	scripts/train/train_anomaly.py	models/anomaly_model.pkl
index	scripts/rag/build_index.py	artifacts/vector_index/
evaluate	scripts/eval/run_eval.py	artifacts/eval_metrics.json

2.3 LangGraph State Machine

Each node receives the full **AgentState** TypedDict and returns only the fields it modifies. LangGraph merges the returned dict into the state before passing it to the next node.

Node	Input fields	Output fields	Model/Tool used
parse	incident	query, log_lines	—
anomaly	log_lines	anomaly_result	IsolationForest + Drain3
retrieve	query	retrieved_chunks	FAISS + BM25
draft	anomaly_result, retrieved_chunks, incident	draft_response	Qwen2.5:7B (Ollama)
validate	draft_response, retrieved_chunks	final_response	safety.py rules

3. Models & Algorithms

3.1 Anomaly Detection — IsolationForest

OpsPilot does **not** fine-tune the LLM. Instead it trains a classical **IsolationForest** on log features extracted by Drain3.

- **Drain3**: streaming log parser that mines templates (e.g. 'ERROR disk usage at <NUM>%'). Trained on 50 k+ HDFS log lines.
- **Features (15-dim vector)**: TF-IDF weights for top vocabulary tokens + statistical features (log line count, unique template count, error/warn ratios).
- **IsolationForest**: contamination=0.01, n_estimators=100. Raw score remapped to [0, 1]: score = $1 - (\text{raw} + 0.5)$.
- **Inference**: given N log lines, Drain3 extracts templates → features built → model returns anomaly score + top templates.

3.2 Hybrid RAG — FAISS + BM25

The RAG index is built once offline and served at inference time.

Component	Details
Corpus	200+ Kubernetes runbooks (Markdown, chunked by section)
Embedding model	sentence-transformers/all-MiniLM-L6-v2 (384-dim)
Vector store	FAISS IndexFlatIP (cosine via normalised vectors)
Keyword store	BM25 (rank-bm25 library)
Fusion formula	score = $0.6 \times \text{FAISS_score} + 0.4 \times \text{BM25_score}$
Top-k retrieved	6 chunks per query
Index path	artifacts/vector_index/

3.3 LLM — Qwen2.5:7B

Qwen2.5:7B was chosen over Llama3 for the following reasons:

Criterion	Qwen2.5:7B	Llama3:8B
Structured JSON output	Excellent — follows schema reliably	Inconsistent — often adds prose
Instruction following	Strong	Moderate
Quantisation	Q4_K_M (~4.4 GB VRAM)	Q4_K_M (~4.7 GB VRAM)
Ollama support	Native	Native
Code fence wrapping	Wraps in ```json (handled)	Varies
GPU throughput (H100)	~800 tokens/s	~750 tokens/s

Note: OpsPilot calls Qwen2.5:7B at inference time only via the Ollama HTTP API. No fine-tuning is performed.

4. Infrastructure & Services

Service	Port	Technology	Status
FastAPI REST API	8000	FastAPI + uvicorn	Running
Streamlit UI	8502	Streamlit 1.58	Running
MLflow Tracking	5001	MLflow 2.x + SQLite	Running
Prometheus	9092	prom/prometheus:v2.53.1	Running (Docker)
Grafana	3001	grafana/grafana:11.1.0	Running (Docker)
Ollama (GPU)	11436	ollama/ollama + nvidia runtime	Running (Docker, H100)
Ollama (shared)	11434	ollama/ollama (CPU only)	Running (Docker, no GPU)

4.1 GPU Ollama Setup

The shared Ollama container (port 11434) uses `--runtime=runc` — no CUDA. OpsPilot runs its own private container with `--runtime=nvidia` on GPU 1 (H100 80 GB):

```
docker run -d \
  --runtime=nvidia \
  --gpus device=1 \
  -v /nuvodata/docker/volumes/ollama/_data:/root/.ollama:ro \
  -p 11436:11434 \
  --name opspilot-ollama \
  ollama/ollama
```

The model volume is mounted read-only from the shared Ollama storage — no re-download needed. CPU inference takes 3+ hours for a 1600-token prompt; GPU inference takes ~30 seconds.

4.2 Prometheus Metrics

The FastAPI app exposes `/metrics` via `prometheus-fastapi-instrumentator`. Key metrics:

Metric	Type	Description
http_request_duration_seconds	Histogram	Request latency per handler (P50/P95/P99)
http_requests_total	Counter	Total requests by handler + status code

Grafana dashboard: 'OpsPilot — Latency & Traffic' auto-provisioned at startup. Shows P50/P95/P99 for `/incident/analyze`, requests/sec, and error rate.

5. API Reference

Full interactive docs: <http://localhost:8000/docs>

Method	Endpoint	Description
GET	/health	Health check — returns status + version
POST	/incident/analyze	Run full 5-node agent pipeline on an incident
POST	/rag/search	Hybrid RAG search (no LLM)
POST	/anomaly/score	Score log lines with IsolationForest
POST	/feedback	Submit feedback on a response
POST	/admin/reload	Hot-reload models without restart
GET	/metrics	Prometheus metrics endpoint

5.1 POST /incident/analyze — Request

```
{
  "incident_id": "INC-2026-0001",
  "alert_title": "NodeDiskRunningFull",
  "service": "payment-api",
  "environment": "production",
  "log_lines": [
    "ERROR disk usage at 95% on /dev/sda1",
    "WARN inode exhaustion detected",
    "ERROR unable to write to disk: no space left"
  ]
}
```

5.2 POST /incident/analyze — Response

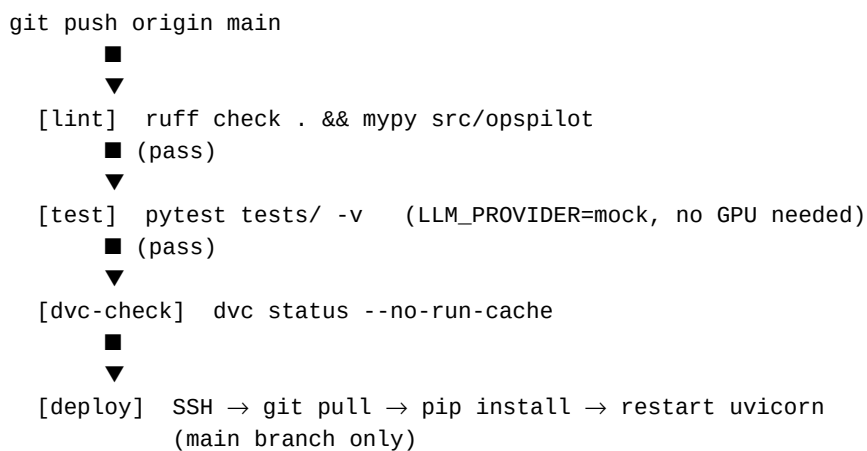
```
{
  "summary": "Disk usage critically high at 95%...",
  "anomaly_report": {
    "score": 0.4796,
    "top_templates": ["ERROR disk usage at <NUM>% on dev sda1", ...]
  },
  "retrieved_context": [
    {"doc_id": "runbook:KubeNodeUnreachable:0", "score": 0.60, ...}
  ],
  "actions": [
    {
      "action": "Drain and cordon the affected node",
      "evidence_doc_ids": ["runbook:KubeNodeUnreachable:0"],
      "rationale": "..."
    }
  ],
  "verification_steps": ["kubectl get pods -o wide ..."],
  "fallback_plan": ["kubectl drain <node> --force ..."],
  "postmortem_markdown": "## Incident Summary\n...",
  "trace": { "nodes_executed": ["parse", "anomaly", "retrieve", "draft", "validate"] }
}
```

6. CI/CD Pipeline

Three GitHub Actions workflows in `.github/workflows/`:

Workflow	Trigger	Jobs
ci.yml	Push to main/dev, PRs to main	lint (ruff+mypy) → test (pytest) → dvc-check → deploy (SSH, main only)
docker-build.yml	Push/PR touching docker/, src/, pyproject.toml	Build api.Dockerfile + ui.Dockerfile in parallel
latency-report.yml	Daily 08:00 UTC + manual	Start API (mock), fire 10 requests, fail if P95 > 2000ms

6.1 CI Flow Diagram



6.2 Required GitHub Secrets (for deploy)

Secret	Value
DEPLOY_HOST	103.42.51.191 (server IP)
DEPLOY_USER	adarshk
DEPLOY_SSH_KEY	Private SSH key (<code>~/.ssh/id_rsa</code> content)

7. DVC Pipeline

DVC manages the full ML pipeline for reproducibility. All stages are defined in **dvc.yaml** with parameters in **params.yaml**.

7.1 Running the Pipeline

```
cd /nuvodata/User_data/adarshk/work/OpsPilot

dvc repro          # run all stages (skips up-to-date stages)
dvc status         # check what's out of date
dvc metrics show   # print MRR, Recall@6, anomaly_pct
```

7.2 Key Metrics (current run)

Metric	Value	Description
MRR	0.59	Mean Reciprocal Rank — RAG retrieval quality
Recall@6	0.58	Fraction of gold docs in top-6 results
anomaly_pct	25%	Fraction of log sequences flagged as anomalous
n_templates	varies	Unique Drain3 templates mined from training set

8. Local Development Setup

8.1 Install

```
git clone https://github.com/adarshmishra121/OpsPilot.git
cd OpsPilot
pip install -e "[api,ui]"
cp .env.example .env # set LLM_PROVIDER=mock for testing
```

8.2 Start Services (development)

```
# FastAPI
uvicorn opspilot.api.main:app --reload --port 8000

# Streamlit UI
streamlit run ui/streamlit_app.py --server.port 8502

# MLflow
python -m mlflow server --backend-store-uri sqlite:///mlflow.db \
    --default-artifact-root ./mlruns --port 5001

# Prometheus (Docker)
docker run -d --name opspilot-prometheus \
    --add-host=host.docker.internal:host-gateway \
    -v ./docker/prometheus.yml:/etc/prometheus/prometheus.yml:ro \
    -p 9092:9090 prom/prometheus:v2.53.1

# Grafana (Docker)
docker run -d --name opspilot-grafana -p 3001:3000 \
    -e GF_SECURITY_ADMIN_PASSWORD=admin grafana/grafana:11.1.0
```

8.3 Run Tests

```
LLM_PROVIDER=mock pytest tests/ -v
# Expected: all tests pass without GPU or Ollama
```

8.4 Environment Variables (.env)

Variable	Default	Description
LLM_PROVIDER	mock	'mock' or 'ollama'
OLLAMA_HOST	http://localhost:11436	Ollama API base URL
OLLAMA_MODEL	qwen2.5:7b	Model name in Ollama
VECTOR_INDEX_PATH	artifacts/vector_index	Path to FAISS+BM25 index
MLFLOW_TRACKING_URI	http://localhost:5001	MLflow server URL

9. Project File Structure

```
OpsPilot/
├── .github/workflows/
│   ├── ci.yml                # lint → test → deploy
│   ├── docker-build.yml      # build api + ui images
│   └── latency-report.yml     # daily P95 latency check
├── artifacts/
│   ├── vector_index/         # FAISS + BM25 index (built by DVC)
│   ├── anomaly_vocab.json     # Drain3 vocabulary
│   ├── train_metrics.json    # training metrics
│   └── eval_metrics.json     # RAG evaluation metrics
├── data/
│   ├── raw/runbooks/         # 200+ Kubernetes Markdown runbooks
│   └── eval/rag_gold.jsonl    # gold-standard RAG eval set
├── docker/
│   ├── api.Dockerfile        # FastAPI container
│   ├── ui.Dockerfile         # Streamlit container
│   └── prometheus.yml        # Prometheus scrape config
├── docs/
│   ├── system_design.md      # ADRs + architecture decisions
│   └── demo_script.md        # Demo Q&A guide
├── models/
│   └── anomaly_model.pkl      # trained IsolationForest
├── src/opsilot/
│   ├── agent/
│   │   ├── graph.py          # LangGraph state machine
│   │   ├── tools.py          # LLM, RAG, anomaly tool functions
│   │   ├── prompts.py        # SYSTEM_PROMPT template
│   │   └── safety.py          # groundedness validator
│   ├── api/
│   │   ├── main.py           # FastAPI app factory
│   │   └── routes/           # health, incident, rag, anomaly, feedback, admin
│   ├── anomaly/              # IsolationForest train + infer
│   ├── rag/                  # FAISS store, BM25, hybrid retriever
│   └── observability/         # Prometheus metrics, structured logging
├── tests/
│   ├── test_agent_safety.py   # groundedness unit tests
│   └── test_api_contract.py   # API contract tests (mocked LLM)
├── ui/streamlit_app.py       # Streamlit incident console
├── dvc.yml                   # DVC pipeline definition
├── params.yaml               # Pipeline hyperparameters
└── docker-compose.yml        # Full stack compose
```

10. Troubleshooting

Problem	Cause	Fix
LLM_PROVIDER stays 'mock'	load_dotenv() called after module-level imports read env vars	main.py calls load_dotenv() before other local imports
Ollama returns 500 after 5 min	CPU-only container: Ollama queue timeout on long prompts	Use --runtime=nvidia GPU container (port 11436)
json.JSONDecodeError in draft_node	Qwen2.5:7B wraps JSON in ```json fences	graph.py strips code fences before json.loads()
SYSTEM_PROMPT KeyError	Literal {} in prompt template treated as .format() placeholders	prompts.py escapes all literal braces as {{ }}
localhost refused in browser	Services run on remote server, firewall blocks ports	SSH tunnel: ssh -L 3001:localhost:3001 ... or VS Code port forwarding
sklearn version warning	Model saved with 1.8.0, running 1.7.2	Retrain with current sklearn: dvc repro